

VETORES atv vetores dia 29/09/2026

Faça um código assembly que execute as seguintes operações:

- Declare três vetores com 10 posições de 8 bits (A1, A2 e A3) e um vetor de 3 posições de 8 bits (A4).
 - *Dica: utilize .DSEG e ".BYTE 8"*
- Inicialize os vetores A1 e A2 com os valores de 1 até 10.
 - *Dica: utilize endereçamento indireto com pós incremento.*
 - *Dica: Utilize um loop com iterações.*
- Some a primeira posição do A1 com a última do A2 e armazene na primeira do A3. Faça isso sucessivamente até que todas as posições sejam operadas.
 - *Dica: utilize endereçamento indireto com pós incremento para acessar o A1 e o A3 e endereçamento indireto com pré decremento para acessar o A2.*
 - *Dica: Utilize um loop com iterações.*
- Some A2(1) e A3(3), A2(3) e A3(4), A2(5) e A3(7) e salve consecutivamente no A4.
 - *Dica: utilize endereçamento indireto com deslocamento para acessar o A2 e o A3 e endereçamento indireto com pós incremento para acessar o A4.*
 - *Dica: Não utilize loop.*

.DSEG ;CRIA O SEGUIMENTO DE DADOS

.ORG SRAM_START

A1: .BYTE 10

A2: .BYTE 10

A3: .BYTE 10

A4: .BYTE 3

.CSEG

setup:

;Inicialize os vetores A1 e A2 com os valores de 1 a 10

LDI R16, 1

LDI YH, HIGH(A1)

LDI YH, LOW(A1)

LDI ZH, HIGH(A2)

LDI ZH, LOW(A2)

init_a1_a2:

ST Y+, R16

ST Z+, R16

```
inc R16
cpi R16, 11 ; é 11 aqui pq ele carrega de 1 até 10, se ele não for igual a 11 ele continua
brne init_a1_a2

rjmp setup
```

SEGUNDA PARTE

Some a primeira posição do A1 com a última do A2 e armazene na primeira do A3. Faça isso sucessivamente até que todas as posições sejam operadas.

- *Dica: utilize endereçamento indireto com pós incremento para acessar o A1 e o A3 e endereçamento indireto com pré decremento para acessar o A2.*
- *Dica: Utilize um loop com iterações.*

```
LDI R16, 1
LDI YH, HIGH(A1)
LDI YL, LOW(A1)
LDI ZH, HIGH(A2 + 10) ;
LDI ZL, LOW(A2 + 10)
```

```
sum_seq:
    LD R17, Y+
    LD R18, -Z
    add R17, r18
    ST X+, R17

    inc r16
    cpi r16, 11
    brne sum_seq
```

- *Some A2(1) e A3(3), A2(3) e A3(4), A2(5) e A3(7) e salve consecutivamente no A4.*

```
LDI YH, HIGH(A1)
LDI YL, LOW(A1)
LDI ZH, HIGH(A2)
LDI ZL, LOW(A2)
LDI XH, HIGH(A4)
LDI XL, LOW(A4)
```

```
LDD R17, Y+1
LDD R18, Z+3
add r17, r18
ST X+, R17
```

```
LDD R17, Y+3
LDD R18, Z+4
add r17, r18
ST X+, R17
```

```
LDD R17, Y+5
LDD R18, Z+7
add r17, r18
ST X+, R17
```

```
rjmp setup
```

=====

```
DSEG          ; Seleciona o segmento de memória de dados (SRAM)
.ORG SRAM_START ; Define o endereço inicial da SRAM (0x0100 para
ATmega328P)
```

```
; Declaração de espaço na SRAM para os vetores
```

```
A1: .BYTE 10      ; Aloca 10 bytes para A1 (endereços 0x0100 a 0x0109)
A2: .BYTE 10      ; Aloca 10 bytes para A2 (endereços 0x010A a 0x0113)
A3: .BYTE 10      ; Aloca 10 bytes para A3 (endereços 0x0114 a 0x011D)
A4: .BYTE 3       ; Aloca 3 bytes para A4 (endereços 0x011E a 0x0120)
```

```
.CSEG          ; Seleciona o segmento de memória de código
```

```
; Configura o ponteiro X para apontar para o início do vetor A1 (0x0100)
LDI XL, LOW(A1)   ; Carrega o byte inferior do endereço de A1 em XL (R26)
LDI XH, HIGH(A1)  ; Carrega o byte superior do endereço de A1 em XH (R27) -> X
aponta para A1[0]
```

```
; Configura o ponteiro Y para apontar para o início do vetor A2 (0x010A)
LDI YL, LOW(A2)   ; Carrega o byte inferior do endereço de A2 em YL (R28)
LDI YH, HIGH(A2)  ; Carrega o byte superior do endereço de A2 em YH (R29) -> Y
aponta para A2[0]
```

```
LDI R16, 1        ; R16 = 1 (Valor inicial a ser gravado nos vetores)
LDI R17, 10       ; R17 = 10 (Contador de iterações do loop)
```

```
loop1:
```

```
ST X+, R16        ; Armazena R16 no endereço apontado por X e incrementa X (+1).
                  ; -> Após executar: X aponta para o próximo elemento de A1.
```

```
ST Y+, R16        ; Armazena R16 no endereço apontado por Y e incrementa Y (+1).
                  ; -> Após executar: Y aponta para o próximo elemento de A2.
```

```
INC R16           ; Incrementa R16 (R16 passa de 1 para 2, 3... até 10)
DEC R17           ; Decrementa o contador de iterações (R17 = R17 - 1)
```

BRNE loop1 ; Salta para 'loop1' se R17 != 0 (repetido 10 vezes)

; Reposiciona o ponteiro X no início de A1

LDI XL, LOW(A1) ; XL recebe a parte baixa do endereço de A1

LDI XH, HIGH(A1) ; XH recebe a parte alta do endereço de A1

; Aponta o ponteiro Y para 1 posição APÓS o fim do vetor A2 (endereço A2 + 10 bytes)

LDI YL, LOW(A2 + 10) ; YL recebe a parte baixa do endereço final de A2

LDI YH, HIGH(A2 + 10) ; YH recebe a parte alta do endereço final de A2

; Configura o ponteiro Z para o início de A3

LDI ZL, LOW(A3) ; ZL recebe a parte baixa do endereço de A3

LDI ZH, HIGH(A3) ; ZH recebe a parte alta do endereço de A3

LDI R17, 10 ; Reinicia o contador de iterações para 10

loop2:

LD R16, X+ ; Lê o valor de A1 no registo R16 e incrementa X (+1).

; -> X avança do início (A1[0]) para o fim (A1[9]).

LD R18, -Y ; Decrementa Y (-1) PRIMEIRO e depois lê o valor de A2 para R18.

; -> Na 1ª vez, Y recua de A2[10] para A2[9] (última posição de A2).

; -> Nas vezes seguintes, recua até A2[0] (primeira posição).

ADD R16, R18 ; R16 = R16 + R18 (Soma o elemento de A1 com o elemento
inverso de A2)

ST Z+, R16 ; Escreve o resultado no endereço apontado por Z (A3) e incrementa
Z (+1).

; -> Z avança do início (A3[0]) para o fim (A3[9]).

DEC R17 ; Decrementa o contador do loop

BRNE loop2 ; Salta para 'loop2' se R17 != 0

; Reinicializa os ponteiros na base de cada vetor

LDI YL, LOW(A2) ; YL recebe a parte baixa de A2

LDI YH, HIGH(A2) ; YH recebe a parte alta de A2

LDI ZL, LOW(A3) ; ZL recebe a parte baixa de A3

LDI ZH, HIGH(A3) ; ZH recebe a parte alta de A3

LDI XL, LOW(A4) ; XL recebe a parte baixa de A4

LDI XH, HIGH(A4) ; XH recebe a parte alta de A4

; --- Operação 1: A2(1) + A3(3) -> A4[0] ---

LDD R16, Y+1 ; Lê A2 com deslocamento +1 (posição índice 1) para R16. (Y
permanece em A2[0])

LDD R18, Z+3 ; Lê A3 com deslocamento +3 (posição índice 3) para R18. (Z
permanece em A3[0])
ADD R16, R18 ; R16 = A2[1] + A3[3]
ST X+, R16 ; Guarda o resultado em A4[0] e incrementa X (+1). -> X aponta para
A4[1]

; --- Operação 2: A2(3) + A3(4) -> A4[1] ---
LDD R16, Y+3 ; Lê A2 com deslocamento +3 (posição índice 3) para R16. (Y
permanece em A2[0])
LDD R18, Z+4 ; Lê A3 com deslocamento +4 (posição índice 4) para R18. (Z
permanece em A3[0])
ADD R16, R18 ; R16 = A2[3] + A3[4]
ST X+, R16 ; Guarda o resultado em A4[1] e incrementa X (+1). -> X aponta para
A4[2]

; --- Operação 3: A2(5) + A3(7) -> A4[2] ---
LDD R16, Y+5 ; Lê A2 com deslocamento +5 (posição índice 5) para R16. (Y
permanece em A2[0])
LDD R18, Z+7 ; Lê A3 com deslocamento +7 (posição índice 7) para R18. (Z
permanece em A3[0])
ADD R16, R18 ; R16 = A2[5] + A3[7]
ST X+, R16 ; Guarda o resultado em A4[2] e incrementa X (+1). -> X aponta para
A4[3] (fim de A4)

end:

RJMP end ; Loop infinito para encerrar a execução do programa